



Voice-First Agentic Commerce

GiftAI — Where Conversation Becomes Commerce

Expert Panel Brainstorm & Product Requirements Document

Prepared by: GiftAI Product & Engineering Panel

PM: Priya Shankar (ex-Shopify, Walmart) · Design: Marcus Chen (ex-Amazon) · Engineering: Dr. Elena Volkov (ex-Google)

Version 1.0 — February 16, 2026

Prepared for: Jaideep Cherukuri, Founder — Halofy / Nortsta

Voice-First Agentic Commerce: Brainstorm & PRD

GiftAI — Where Conversation Becomes Commerce

PART 1: EXPERT PANEL BRAINSTORM

Panel Members

- **Priya Shankar (PM)** — 12 years product at Shopify (Commerce Platform) → Walmart (Voice Commerce Pilot). Led Shopify's conversational commerce beta and Walmart's voice-ordering initiative.
 - **Marcus Chen (Design Head)** — 14 years UX at Amazon (Alexa Shopping, Amazon Go). Led the multimodal shopping design language for Alexa with screen.
 - **Dr. Elena Volkov (Principal Systems Engineer)** — 11 years at Google (Search, Assistant, Gemini integration). Architected the real-time function-calling pipeline for Google Assistant's commerce flows.
-

Round 1: The Core Problem

Priya (PM): Let me frame where we are. We have a working text-based gift commerce agent — product search, recommendations, cart, checkout — all through chat. Separately, there's a LiveKit + Gemini voice agent that does real-time speech-to-speech with function calling. The founder's vision: voice as the *primary driver*, with text streaming alongside as visual feedback, and product cards appearing in context. The question is — how do we merge these without losing what already works?

Marcus (Design): I've seen this exact pattern fail three times at Amazon. The failure mode is always the same: teams bolt voice onto a visual UI and end up with something that's bad at both. Voice-first doesn't mean "voice-only." At Alexa Shopping, our breakthrough was understanding that *voice is the navigation layer, screens are the confirmation layer*. You speak to browse and decide; you see to verify and act.

Dr. Elena (Eng): That's the right mental model. From a systems perspective, the key constraint is latency. Gemini's native audio model does speech-to-speech in ~300ms. But if we're also doing tool calls (searching Shopify, fetching products), the user hears silence while the backend works. At Google Assistant, we solved this with "filler behaviors" — the agent acknowledges immediately ("Let me find some options...") while the tool call executes in parallel. The architecture has to support this.

Priya: Exactly. And there's a business constraint too. At Walmart, we found that voice-only shoppers had a 73% cart abandonment rate. Why? Because they couldn't see what they were buying. The moment we added visual confirmation — showing the product image while the voice described it — completion rates jumped to near-parity with visual shopping. Voice accelerates discovery, but visual seals the deal.

Marcus: So the design principle is clear: **Voice drives. Visuals confirm. Touch completes.** The user says "show me wedding gifts under 300 dollars." The voice immediately says "I found 5 beautiful wedding gifts, let me show you." Product cards slide in. The user can say "tell me more about the second one" OR tap the card. Both paths work.

Round 2: The Interaction Model Debate

Marcus: Here's where I want to push back on a purely linear chat model. If we stream voice-to-text into a traditional chat interface, we get a wall of text that's hard to scan. At Amazon, we experimented with three patterns for voice + visual commerce:

1. **Chat Transcript Model** — Voice streams as text bubbles. Products appear inline. Basically what exists now but with voice input.
2. **Stage + Transcript Model** — A "stage" area (top 60%) shows the active content (products, cart, checkout). Below it, a compact transcript shows the conversation. Voice controls float.
3. **Ambient Canvas Model** — The entire screen is the product canvas. Voice conversation floats as subtle captions/overlays. Minimal chrome.

Pattern 2 won every user test by a 2:1 margin. Pattern 1 felt "chatbot-like." Pattern 3 was beautiful but users felt lost without conversational context.

Dr. Elena: I agree with the Stage model, but with an important architectural consideration. In the current text system, the LLM response and the product data arrive together in a single API call. With voice, we have *two streams*: the audio stream (real-time, continuous) and the data stream (products, cart updates — event-driven, asynchronous). These **MUST** be separate channels. LiveKit gives us this for free — the audio goes through WebRTC tracks, and we can use LiveKit's Data Channel for structured events (products found, cart updated, etc.).

Priya: That's the key insight. Let me map the user journey:

1. **Idle State** → Beautiful ambient screen. Pulsing mic icon. "Say something or type below."
2. **User speaks** → Real-time transcription appears as a caption (not a chat bubble — more like subtitles). Visualizer shows the mic is active.
3. **Agent processing** → Brief "thinking" animation. Voice says filler ("Great choice, let me find those...").
4. **Agent responds with products** → Voice describes recommendations.
SIMULTANEOUSLY, product cards animate into the Stage area. The transcript below captures the conversation.
5. **User browses visually** → Can scroll/tap cards. Can also say "tell me about the rose one" — positional/descriptive references work.
6. **Add to cart** → Voice says "Added! Your cart now has 2 items totaling \$340." Cart indicator updates. Stage can optionally show cart.
7. **Checkout** → Voice says "Ready to checkout? I'll open the Shopify checkout for you." Checkout URL opens.

Marcus: Yes! And critically — the text input bar is ALWAYS there at the bottom. Voice-primary doesn't mean voice-exclusive. Some people will want to type in a noisy office. Some will want to paste a URL. The text input should feel secondary but always accessible. Think of it like search engines: most people type, but voice is there for when it's better.

Round 3: The Visualizer & Emotional Feedback

Marcus: This is where most voice UIs are criminally underdesigned. Voice is *intimate*. When someone talks to your app, they need feedback that feels human — not a spinning circle. Let me describe the visual language I envision:

The Orb. Center of the stage when no products are shown. It's a fluid, animated orb (think: a glowing sphere that breathes). It has states: - **Idle:** Soft, slow breathing pulse. Purple/brand gradient. - **Listening:** Reacts to the user's voice amplitude. Ripples outward. Color shifts to brighter. - **Thinking:** Contracts slightly, particles orbit. Subtle shimmer. - **Speaking:** Expands, rhythmic pulsation matching the agent's speech amplitude. Color warms.

When products appear, the Orb gracefully shrinks and docks to the top-left of the stage — it stays visible as a presence indicator, but the products take center stage.

Dr. Elena: That's beautiful UX, but let me flag the engineering cost. Real-time audio visualization at 60fps requires either WebAudio API analysis (which we already get from LiveKit) or a shader-based approach. The BarVisualizer in the current LiveKit repo is CPU-based. For the Orb, I'd recommend a lightweight WebGL shader — something like a displaced sphere with perlin noise, driven by the audio RMS value from LiveKit's audio track. It's ~50 lines of shader code and runs entirely on the GPU.

Priya: From a product perspective, the Orb serves another purpose: trust signaling. Users need to know when they're being heard, when the AI is "thinking," and when it's safe to speak again. At Walmart, our biggest voice commerce complaint was "I don't know when it's my turn to talk." The Orb's state changes solve that.

Marcus: And for voice transcription — I want real-time streaming text that appears word-by-word as the user speaks and as the agent speaks. Like live captions. Not after-the-fact. This creates the feeling of a synchronized conversation.

Dr. Elena: LiveKit provides transcription events through their agents SDK. The Python agent can emit `TranscriptionSegment` events that the frontend receives through the `useVoiceAssistant` hook. We can get word-level timing for agent speech. For user speech, Gemini's native audio doesn't output text transcripts in the audio modality — we'd need to add a parallel STT stream or switch to TEXT+AUDIO modality to get transcripts. That's a design tradeoff.

Priya: We need the transcription. Users said it at Walmart, they said it at Shopify: "I want to see what I said." It's accountability. It's confirmation. And it's accessibility. Some users are hard of hearing and still want voice UX.

Dr. Elena: Then the architecture is: Gemini runs in AUDIO+TEXT modality so we get both the voice response AND the text response. The text streams to the UI as captions. The audio plays through LiveKit. They're synchronized by timestamps.

Round 4: Product Display & Commerce Flow

Priya: Let's talk about the money flow. How does "I want that rose box in the crystal variant" become a Shopify checkout?

Dr. Elena: In the current text system, the server-side Gemini call includes function definitions (search_products, add_to_cart, etc.), and Gemini returns function calls that the server executes. The response includes both text and structured product data.

With the LiveKit agent, it's different. The Python agent has tool functions registered on it. When the user says "show me wedding gifts," Gemini's realtime model will call the search tool, the Python agent executes it, returns the results to Gemini, and Gemini speaks the response. The frontend needs to receive the tool call results to display product cards.

Here's my proposed event flow: 1. User speaks → LiveKit → Python Agent → Gemini receives audio 2. Gemini decides to call `search_products` tool 3. Python agent intercepts the tool call, executes Shopify MCP/API search 4. Python agent sends structured data to frontend via LiveKit Data Channel: `{ type: "products", data: [...products] }` 5. Python agent returns results to Gemini, Gemini generates voice response 6. Frontend receives data event → renders product cards in Stage 7. Frontend receives audio → plays agent voice + shows transcript

The critical point: steps 4 and 5 happen in PARALLEL. The frontend gets the product data as fast as the API can return it, independent of how long Gemini takes to compose the voice response.

Marcus: Love that. The cards should animate in with a stagger — card 1 slides in, then card 2, card 3 — like the agent is "laying them out on a table." It matches the voice: "Here are some wedding gifts. First, this gorgeous star map..."

Priya: For add-to-cart, the flow should be: - Voice: "Add the star map to my cart" → Agent calls `add_to_cart` tool → Cart state updates via data channel → Frontend shows cart update animation (a subtle "added!" toast + cart badge increment) → Agent voice confirms. - Touch: User taps "Add to Cart" on a card → Same API call → Same feedback. The voice agent acknowledges: "Added! Good choice."

Marcus: Wait — that last point is key. If the user taps a card while the voice agent is active, the agent should know about it. It should feel like a unified conversation, not two separate systems.

Dr. Elena: That's achievable. When the user taps "Add to Cart" on the frontend, we send a data message BACK to the Python agent through LiveKit's data channel: `{ type: "user_action", action: "add_to_cart", product: "Star Map" }`. The agent can then generate a contextual voice response. It's bidirectional.

Round 5: Text Chat Coexistence

Priya: Here's a scenario Jai specifically flagged: the existing text chat must remain fully functional. How do we handle the user typing while voice is active?

Marcus: Two states: 1. **Voice Active** — Mic on, Orb prominent. Text bar is still there but compact. User can type at any time — typing temporarily suppresses the mic (push-to-type behavior). The typed message is sent to the same conversation context. 2. **Voice Inactive** — Mic off. The UI shifts to emphasize the text input. Orb shrinks to a mic button. The experience is basically the current text chat, just prettier.

The key: ONE conversation context, TWO input modalities. Whether the user speaks or types, it goes into the same history, and the agent responds appropriately (voice if mic is on, text if mic is off).

Dr. Elena: Architecturally, this means the frontend maintains a single conversation state. Messages can come from: - Voice (transcribed user speech → rendered as user message) - Text input (typed → rendered as user message) - Agent voice (transcribed agent speech → rendered as assistant message with products/actions) - Agent voice triggers (product data from data channel → rendered as cards)

The conversation state is the same `messages[]` array we already have, just with richer sources.

Priya: And suggestions/quick actions should adapt. When voice is active, show fewer typed suggestions (the user is talking). When voice is off, show the current suggestion chips. Always show context-relevant actions like "Checkout" or "View Cart" as tappable buttons regardless of mode.

Round 6: System Architecture Deep Dive

Dr. Elena: Let me draw the full system architecture. There are four key layers:

Layer 1: Client (Next.js) - LiveKit Room connection (audio/video tracks) - Unified Chat State Manager - Product Stage (carousel, cards, detail views) - Voice Controls (mic, speaker, visualizer) - Text Input (fallback/complement) - Cart Panel + Checkout

Layer 2: LiveKit Cloud - WebRTC signaling - Audio routing (client ↔ agent) - Data channels (structured events)

Layer 3: Python Voice Agent - LiveKit Agents SDK - Gemini 2.5 Flash Realtime (native audio + text) - Tool functions (Shopify MCP bridge) - Data channel emitter (products, cart, actions)

Layer 4: Shopify Backend - MCP server (jaguar-9969) for search/cart/policies - Admin API for product details/inventory - Storefront API for checkout URL generation

The critical path for a voice query:

```
User speaks → [WebRTC] → LiveKit Cloud → [WebRTC] → Python Agent
  → Gemini processes audio → decides tool call → Agent executes
  → [Data Channel] products → Client renders cards (instant)
  → Gemini composes voice → [WebRTC] → Client plays audio (parallel)
```

Marcus: What about the text-only fallback? If LiveKit or the voice agent is down?

Dr. Elena: The existing `/api/chat` route remains. If the LiveKit connection fails or the user turns off voice, the frontend falls back to the current text API. Zero degradation. This is critical for reliability — voice is an enhancement, not a dependency.

Priya: What about the token endpoint? The current voice agent uses a generic room. For commerce, we need per-session isolation.

Dr. Elena: Right. Each user session gets a unique room name: `gift-session-{uuid}`. The token API generates a room-scoped token. The Python agent has `accept_all_jobs` which will spin up a new agent instance per room. Each agent instance maintains its own Shopify session state (cart ID, conversation history).

Round 7: Mobile & Responsive Design

Marcus: Most gift shopping happens on mobile. The voice interface actually has an advantage here — speaking is faster than typing on a phone. But the Stage layout needs to adapt:

Desktop (>1024px): - Left 65%: Stage (products/orb) + transcript below - Right 35%: Cart panel (collapsible) - Bottom: Text input + voice controls

Tablet (768-1024px): - Full width Stage - Cart as slide-over panel - Bottom: Text input + voice controls

Mobile (<768px): - Full width Stage (products stack vertically, carousel for many) - Cart as bottom sheet - Fixed bottom bar: Mic button (large, center) + text input (expands on tap) + cart badge - Transcript as overlay captions (auto-fade)

The Orb on mobile should be smaller — maybe 60px — and sit above the input bar when no products are shown. When products appear, it becomes a 32px indicator in the header.

Priya: One thing from Walmart's mobile voice commerce: the mic button needs to be OBVIOUS. Like, embarrassingly large. Users on mobile don't instinctively look for a mic button in a shopping app. It should be the hero element. Think of it as the "Google Assistant" floating action button — prominent, pulsating, inviting.

Dr. Elena: For mobile, there's a technical consideration: iOS Safari requires user gesture to start audio playback and mic access. The current LiveKit setup handles this with `<StartAudio>`, but we need to make sure the first interaction is a conscious tap on the mic button, not an auto-play attempt.

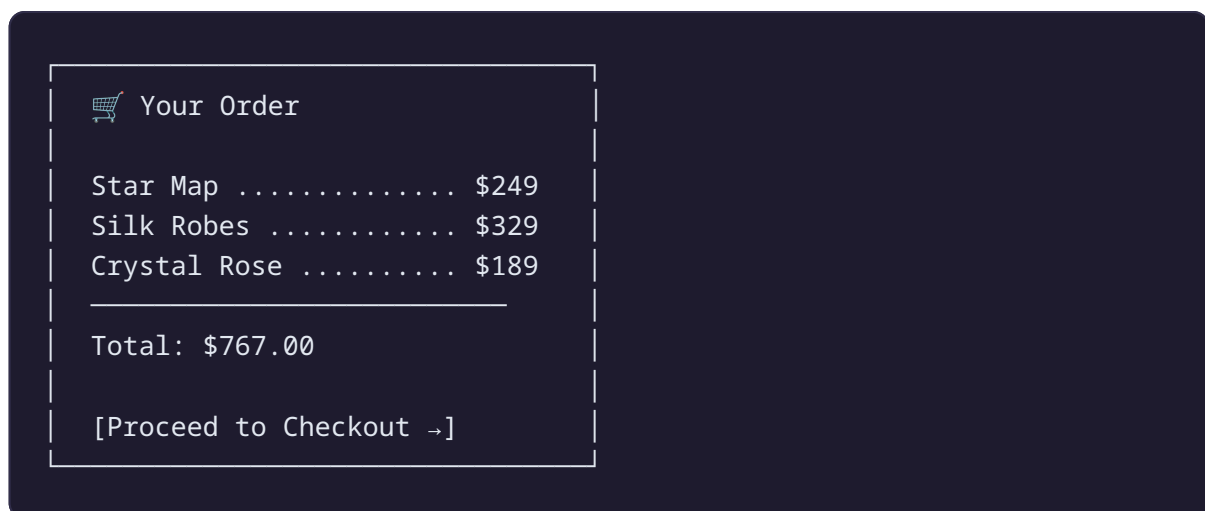
Round 8: Checkout & Critical Commerce Flows

Priya: Checkout is where money changes hands. Let me define the voice checkout flow:

1. User: "I'm ready to checkout"
2. Agent: "You have 3 items in your cart totaling \$547. Would you like to proceed to Shopify checkout?"
3. Stage shows: Order summary card (items, prices, total)
4. User: "Yes" / taps "Proceed to Checkout"
5. Agent: "Opening checkout now. You'll complete payment on Shopify's secure checkout page."
6. Frontend opens checkout URL (new tab on desktop, in-app on mobile)

The voice agent should NEVER handle payment details. It's a legal and trust nightmare. The agent facilitates up to the checkout handoff.

Marcus: The order summary card should be specially designed — not just a list. Think of it as a receipt preview:



Dr. Elena: For the checkout URL, the current system uses Shopify's `/cart/c/{encoded_variants}` URL. This needs to be generated server-side (in the Python agent) when the user confirms. The agent sends it to the frontend via data channel, and the frontend opens it. Same mechanism as products, different event type.

Round 9: What Makes This 10x Better

Priya: Let me list the features that make this not just "voice added" but genuinely 10x:

- 1. **Instant Gratification** — "Show me birthday gifts" → products appear in <2 seconds while the voice explains them. No typing, no waiting for text response.
- 2. **Hands-Free Shopping** — Cooking? Driving? Just talk. "Add the whiskey set." Done.
- 3. **Emotional Connection** — The warm Gemini voice with personality creates affinity that text can't match. Gift shopping is emotional; the agent should feel like a friend helping you pick gifts.
- 4. **Multimodal Intelligence** — "I'm looking for something like the rose one but cheaper." The agent understands the visual context + the spoken request.
- 5. **Accessibility** — Voice input helps users with motor disabilities. Spoken output helps visually impaired users. Transcripts help hearing-impaired users. The multimodal approach is inherently more accessible.
- 6. **Reduced Friction** — At Walmart, voice shoppers who completed the flow had 40% higher basket sizes than text shoppers. Why? Because saying "also add that" is frictionless. Every additional tap/type is a drop-off point.

Marcus: And the UX improvements to existing features: - Product cards get a "voice context" indicator (subtle ear icon) when the agent is currently describing that product - The carousel gets voice-navigable: "scroll right," "show me the next one" - Cart updates have voice confirmation + visual animation — double feedback - The welcome experience is transformed: the Orb pulses, the agent says "Welcome to the gift store! What's the occasion?" — it's inviting, not intimidating

Dr. Elena: From an engineering perspective, the 10x is: - Sub-second tool call responses (LiveKit data channel is ~50ms RTT) - Parallel audio + data streams (no blocking) - Graceful degradation (voice off → full text experience) - Session persistence (refresh doesn't lose context) - The Python agent's tool functions are essentially the same as the MCP tools — we're reusing all the Shopify integration work

Round 10: Risks & Mitigations

Priya: | Risk | Impact | Mitigation | |-----|-----|-----| | Voice recognition errors | Wrong products added | Visual confirmation before cart actions; "Did you mean...?" pattern | | Background noise | False triggers | Push-to-talk option; VAD (Voice Activity Detection) tuning | | API latency spikes | Awkward silence | Filler phrases + loading

shimmer cards | | Privacy concerns | Users uncomfortable with mic | Voice is opt-in; text-first by default; clear mic indicator | | Mobile browser mic issues | Voice doesn't work | Graceful fallback; text chat always available | | Cost (Gemini realtime API) | Per-second billing | Idle timeout; auto-mute after 30s silence |

Dr. Elena: The cost one is real. Gemini's native audio model bills per second of active connection. We should implement: - Auto-disconnect after 2 minutes of silence - Visual indicator showing connection is active - "Reconnect" button if disconnected - Voice sessions should be opt-in, not always-on

Marcus: And we should A/B test: some users might prefer voice off by default. Don't force it. Let the Orb be enticing enough that users WANT to tap it.

PART 2: PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Product Overview

1.1 Vision

Transform GiftAI from a text-based chat store into a **voice-first, visually-rich agentic commerce experience** where users discover, explore, and purchase luxury gifts through natural conversation — spoken or typed.

1.2 Tagline

"Just say what you're looking for."

1.3 Success Metrics

Metric	Current	Target	Measurement
Time to first product view	~8s (type + wait)	<3s (speak + see)	Analytics

Metric	Current	Target	Measurement
Cart completion rate	Baseline	+30%	A/B test
Session duration	Baseline	+50%	Analytics
Items per cart	Baseline	+25%	Shopify data
User satisfaction (NPS)	N/A	>60	Survey

2. User Personas

2.1 "Busy Buyer" (Primary)

- **Who:** 28-45, shopping for gifts on-the-go
- **Context:** On phone, multitasking (commuting, cooking)
- **Need:** Quick voice-driven discovery, minimal taps
- **Quote:** "I need a wedding gift for Saturday, just tell me what's good."

2.2 "Deliberate Explorer" (Secondary)

- **Who:** 25-55, browsing for inspiration
- **Context:** On desktop/tablet, relaxed browsing
- **Need:** Visual browsing with conversational guidance
- **Quote:** "I want to see everything you have, then narrow down."

2.3 "Accessibility User" (Critical)

- **Who:** Any age, motor/visual/hearing impairments
 - **Need:** Multiple modalities for input and output
 - **Quote:** "I can't type easily, but I can describe what I want."
-

3. Feature Specifications

3.1 Voice Engine (P0 — Must Have)

3.1.1 LiveKit Integration

- **Connection:** Auto-join LiveKit room on mic activation
- **Room naming:** `gift-{session-uuid}` per user session
- **Reconnection:** Auto-reconnect on network interruption
- **Fallback:** Full text chat if LiveKit unavailable

3.1.2 Gemini Realtime Agent

- **Model:** `gemini-2.5-flash-native-audio-preview`
- **Modality:** AUDIO + TEXT (for transcription)
- **Voice:** "Puck" (warm, friendly — configurable)
- **Temperature:** 0.7 (creative but reliable)
- **System prompt:** Gift commerce persona with Shopify tool awareness

3.1.3 Tool Functions (Python Agent)

```
@llm.function_tool(description="Search for gift products by query, occasion, budget")
async def search_products(self, query: str, max_price: float = None, occasion: str = None) -> str

@llm.function_tool(description="Add a product variant to the shopping cart")
async def add_to_cart(self, product_title: str, variant_title: str = None) -> str

@llm.function_tool(description="Get detailed information about a specific product")
async def get_product_details(self, product_title: str) -> str

@llm.function_tool(description="Get store policies like shipping, returns, refunds")
async def get_store_policies(self, policy_type: str) -> str

@llm.function_tool(description="View current cart contents and total")
async def view_cart(self) -> str

@llm.function_tool(description="Generate checkout URL for current cart")
async def checkout(self) -> str
```

3.1.4 Data Channel Events

The Python agent emits structured events to the frontend via LiveKit data channel:

```
// Agent → Frontend events
type AgentEvent =
  | { type: 'products_found', products: Product[], query: string }
  | { type: 'product_detail', product: Product }
  | { type: 'cart_updated', cart: CartState }
  | { type: 'checkout_ready', url: string }
  | { type: 'agent_state', state: 'listening' | 'thinking' | 'speaking' | 'idle' }
  | { type: 'transcript', role: 'user' | 'agent', text: string, final: boolean }

// Frontend → Agent events
type UserEvent =
  | { type: 'user_action', action: 'add_to_cart', productId: string, variantId?: string }
  | { type: 'user_action', action: 'view_details', productId: string }
  | { type: 'user_action', action: 'checkout' }
  | { type: 'text_message', content: string }
```

3.2 The Stage — Product Display Area (P0)

3.2.1 Layout States

1. **Empty (Welcome)** — Orb centered, welcome text, suggestion chips
2. **Products** — Horizontal carousel (5+ items) or grid (≤ 4 items). Orb docked top-left.
3. **Product Detail** — Expanded single product view with images, description, variants, add-to-cart
4. **Cart** — Order summary with line items, quantities, total, checkout button
5. **Checkout Confirmation** — Final summary before redirect

3.2.2 Transitions

- Welcome → Products: Cards stagger-animate in from right
- Products → Detail: Selected card expands, others fade
- Products → Cart: Slide from right
- Any → Welcome: Fade out, Orb returns to center

3.3 The Orb — Voice Presence Indicator (P0)

3.3.1 Visual Design

- **Shape:** Fluid sphere, 120px diameter (desktop), 64px (mobile)
- **Rendering:** CSS with layered gradients and filters (performant, no WebGL needed for v1)
- **Colors:** Brand purple gradient base

3.3.2 States

State	Visual	Audio Cue
Idle	Slow breathing pulse, dim	None
Connecting	Quick pulse, brightening	Subtle chime
Listening	Ripples respond to user voice amplitude	None
Thinking	Particles orbit, shimmer	None
Speaking	Rhythmic expansion matching speech	Agent voice plays
Error	Red tint, shake	Error tone

3.3.3 Positioning

- **No products:** Centered in Stage area
- **Products visible:** Docked to conversation area header (32px, inline with status)
- **Mobile:** Above input bar (48px)

3.4 Conversation Transcript (P0)

3.4.1 Layout

- Below the Stage area
- Compact messages (not full chat bubbles — more like a transcript)
- User messages: right-aligned, brand color, prefixed with 🎤 (voice) or ⌨️ (typed)
- Agent messages: left-aligned, with timestamps
- Product references in agent messages are clickable (scroll to card)

3.4.2 Real-Time Streaming

- User speech: Words appear in real-time as user speaks (interim transcript)
- Agent speech: Text streams word-by-word synchronized with voice
- System events: "[Added Star Map to cart]" as subtle inline notifications

3.5 Voice Controls (P0)

3.5.1 Primary Mic Button

- **Position:** Center of bottom control bar
- **Size:** 56px (desktop), 64px (mobile) — the largest interactive element
- **States:** Off (outline), Active (filled, pulsing), Muted (crossed out)
- **Interaction:** Tap to toggle. Long-press for push-to-talk mode.
- **First activation:** Requests mic permission, connects LiveKit

3.5.2 Secondary Controls

- **Speaker toggle:** Mute/unmute agent voice (text still shows)
- **End voice session:** Disconnect from LiveKit, revert to text mode
- **Audio level indicator:** Subtle bar showing mic input level

3.6 Text Input (P0 — Existing, Enhanced)

3.6.1 Behavior

- Always visible at bottom of screen
- When voice is active: Compact, single-line. Typing auto-pauses mic.
- When voice is off: Full chat input (current behavior preserved)
- Supports all existing functionality: search, cart commands, policy questions

3.6.2 Smart Suggestions

- Contextual quick-action buttons above input:
- After products shown: "Add first one", "Tell me more", "Show cheaper options"
- After add-to-cart: "Checkout", "Keep shopping", "View cart"
- After checkout: "Start over", "Share gift"

3.7 Cart Integration (P0 — Existing, Enhanced)

3.7.1 Visual Updates

- Cart badge in header (existing, keep)
- On add-to-cart: Brief "item added" animation (card flies to cart icon)
- Cart panel opens with voice command OR tap

3.7.2 Voice Cart Management

- "What's in my cart?" → Agent reads items + total, Stage shows cart
- "Remove the whiskey" → Agent confirms, cart updates
- "Change the star map to the large variant" → Agent updates

3.8 Enhanced Product Cards (P1 — Improvement)

3.8.1 Voice Context Indicators

- When agent is describing a specific product, that card gets a subtle highlight/glow
- "Currently describing" indicator (small speaker icon on card)

3.8.2 Quick Voice Actions on Cards

- Each card has a floating "  Add" voice shortcut — tap once, agent confirms variant selection by voice
- Tap the card to expand details (existing behavior)

3.9 Mobile Optimizations (P1)

3.9.1 Bottom Sheet Cart

- Cart opens as a swipe-up bottom sheet instead of a side panel
- Drag handle at top for partial/full open

3.9.2 Haptic Feedback

- Vibrate on mic activation
- Subtle haptic on cart addition
- Haptic on successful checkout redirect

3.9.3 Compact Voice Mode

- On mobile, when voice is active and products are showing:
 - Transcript becomes 2-line caption overlay (auto-fades)
 - More screen space for products
 - Mic button and minimal controls at bottom
-

4. User Flows

4.1 Flow 1: Voice Discovery → Purchase

```
[User opens app]
↓
[Stage: Orb (idle) + Welcome message + Suggestions]
↓
[User taps mic button → Orb activates (listening)]
↓
User: "I need a birthday gift for my brother who loves cooking"
↓
[Transcript: Real-time text streaming of user speech]
[Orb: transitions to thinking]
↓
[Agent calls search_products("birthday cooking brother")]
[Data channel → frontend receives products]
↓
[Stage: Product carousel animates in (3-5 cards)]
[Orb: docks to top-left, transitions to speaking]
[Agent voice: "I found some great options for a cooking enthusiast!
Let me show you..."]
[Transcript: Agent text streams alongside voice]
↓
User: "Tell me more about the second one"
↓
[Agent calls get_product_details(product_2)]
[Stage: Card 2 expands to detail view]
[Agent voice describes product in detail]
↓
User: "Add it to my cart, the medium size"
↓
[Agent calls add_to_cart(product_2, "Medium")]
```

```
[Cart badge animates +1]
[Stage: Brief "Added!" animation on card 2]
[Agent voice: "Added! Your cart now has 1 item for $249.
  Want to keep shopping or checkout?"]
  ↓
User: "Let's checkout"
  ↓
[Agent calls checkout()]
[Stage: Order summary card appears]
[Agent voice: "Here's your order summary. $249 total.
  Opening Shopify checkout now."]
  ↓
[Frontend opens checkout URL]
```

4.2 Flow 2: Text Fallback (Voice Off)

```
[User opens app → Orb idle → Types in text bar]
  ↓
[Exactly current behavior – nothing changes]
[Text chat, product cards, cart, checkout – all same]
  ↓
[User can tap mic at any point → voice activates]
[Conversation context transfers seamlessly]
```

4.3 Flow 3: Mixed Modality

```
[User has voice active, browsing products]
  ↓
[User types: "add the $189 one to cart"]
  ↓
[Mic auto-pauses while typing]
[Agent processes as text, responds by voice: "Added the Crystal Rose Box!"]
[Mic resumes after agent finishes speaking]
```

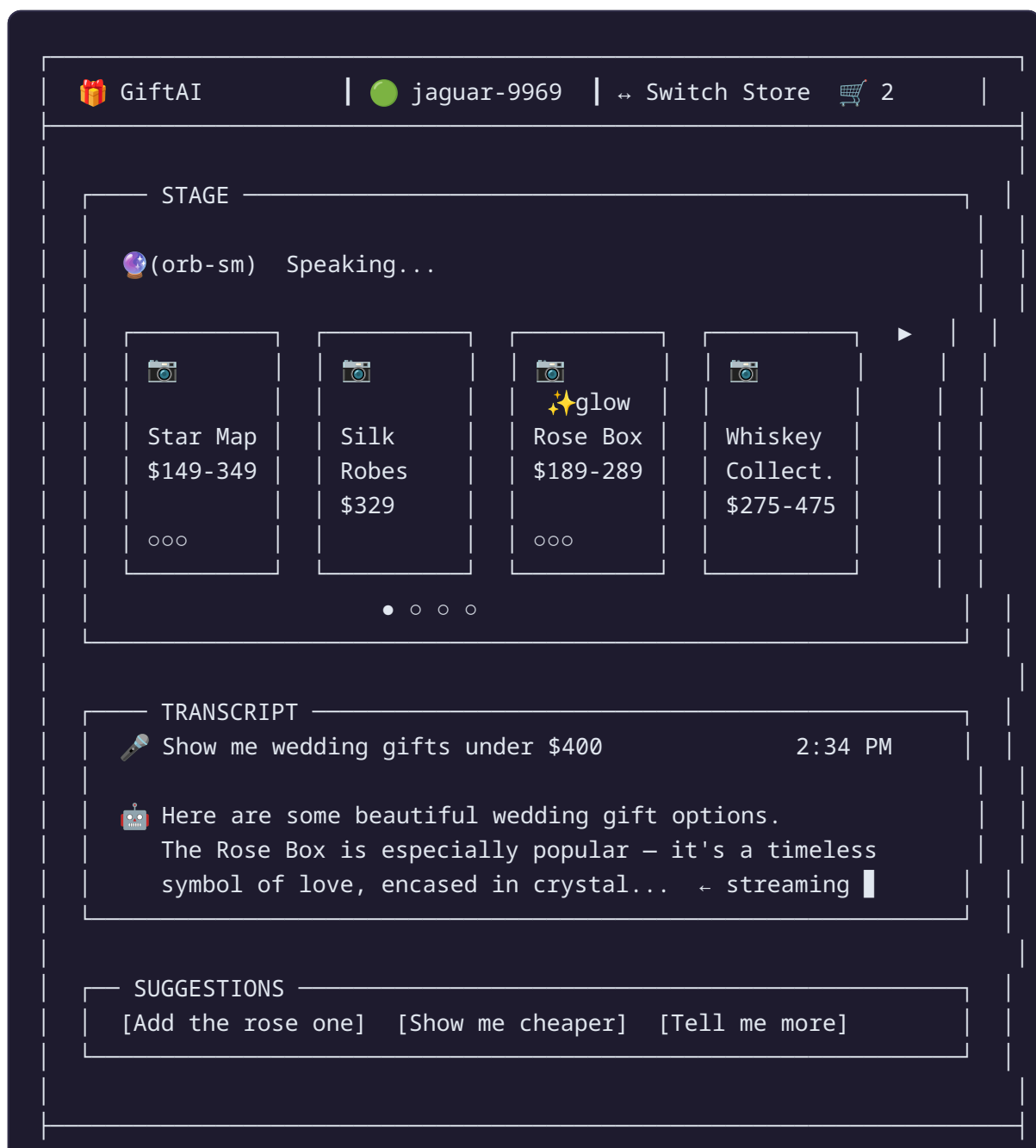
4.4 Flow 4: Store Switching (Existing, Preserved)

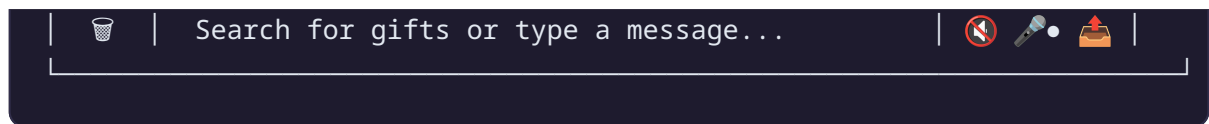
```
[User taps "Switch Store" in header]
  ↓
[Modal opens (existing StoreSwapModal)]
[If voice active: paused during modal]
```

↓
[User enters new store credentials → Connect]
↓
[Chat resets, voice agent reloads with new store context]

5. UI Mockups (Text Wireframes)

5.1 Desktop — Voice Active with Products

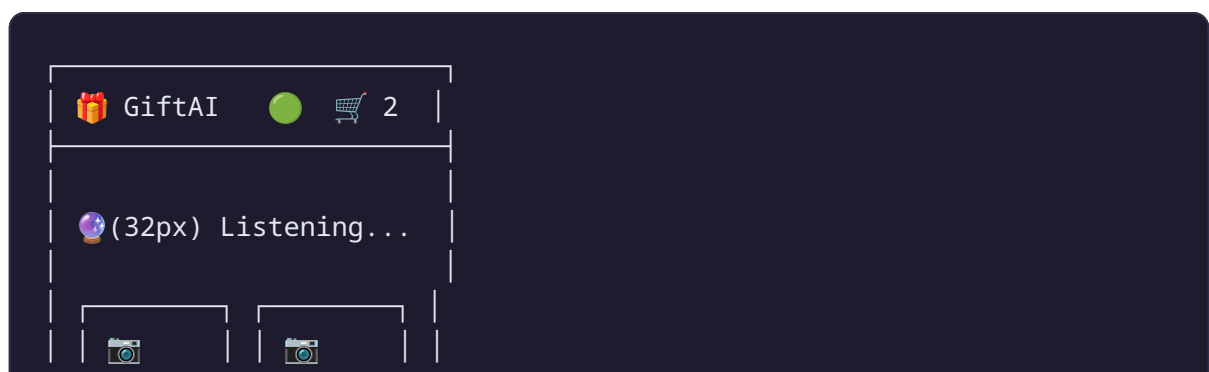


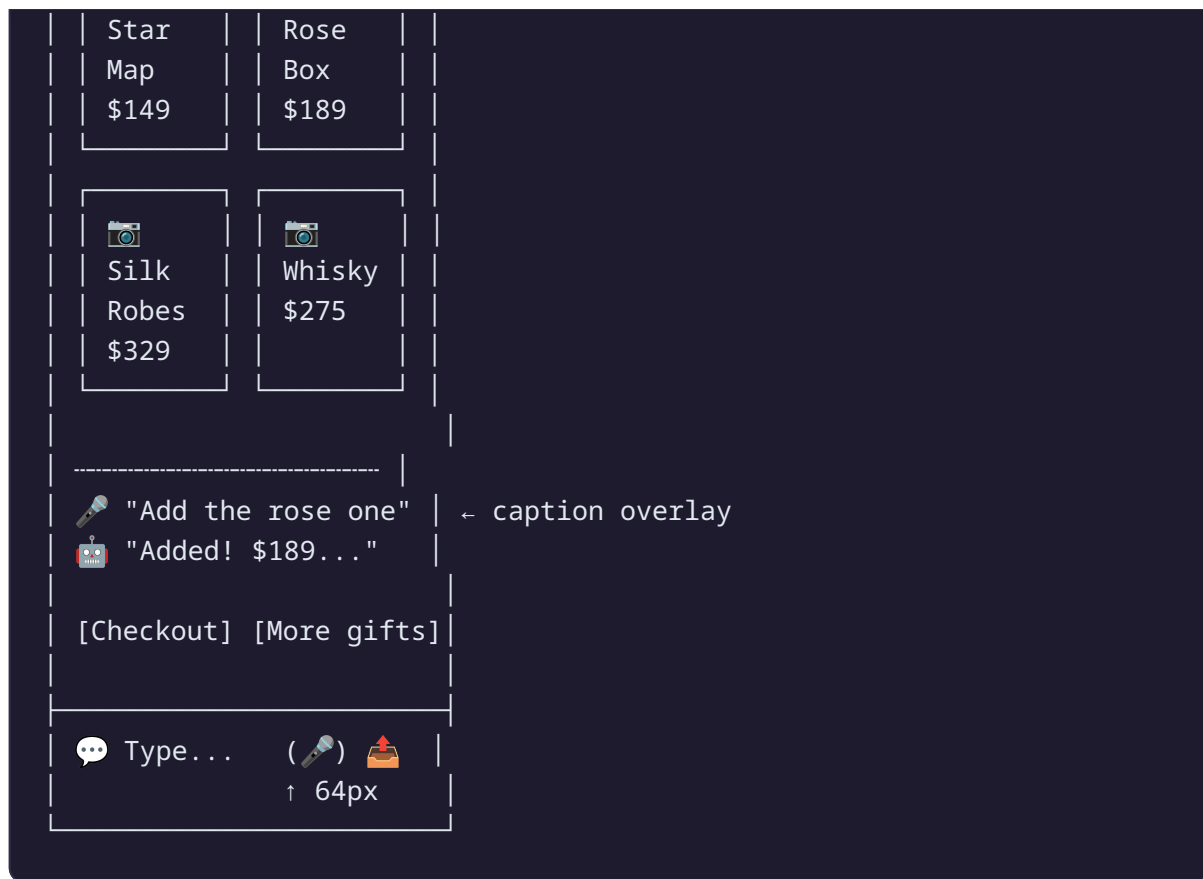


5.2 Desktop — Voice Active, No Products (Welcome/Idle)



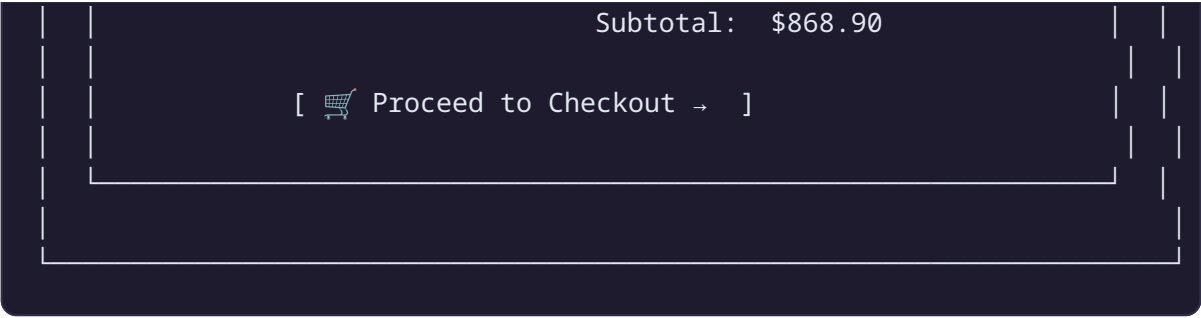
5.3 Mobile — Voice Active with Products





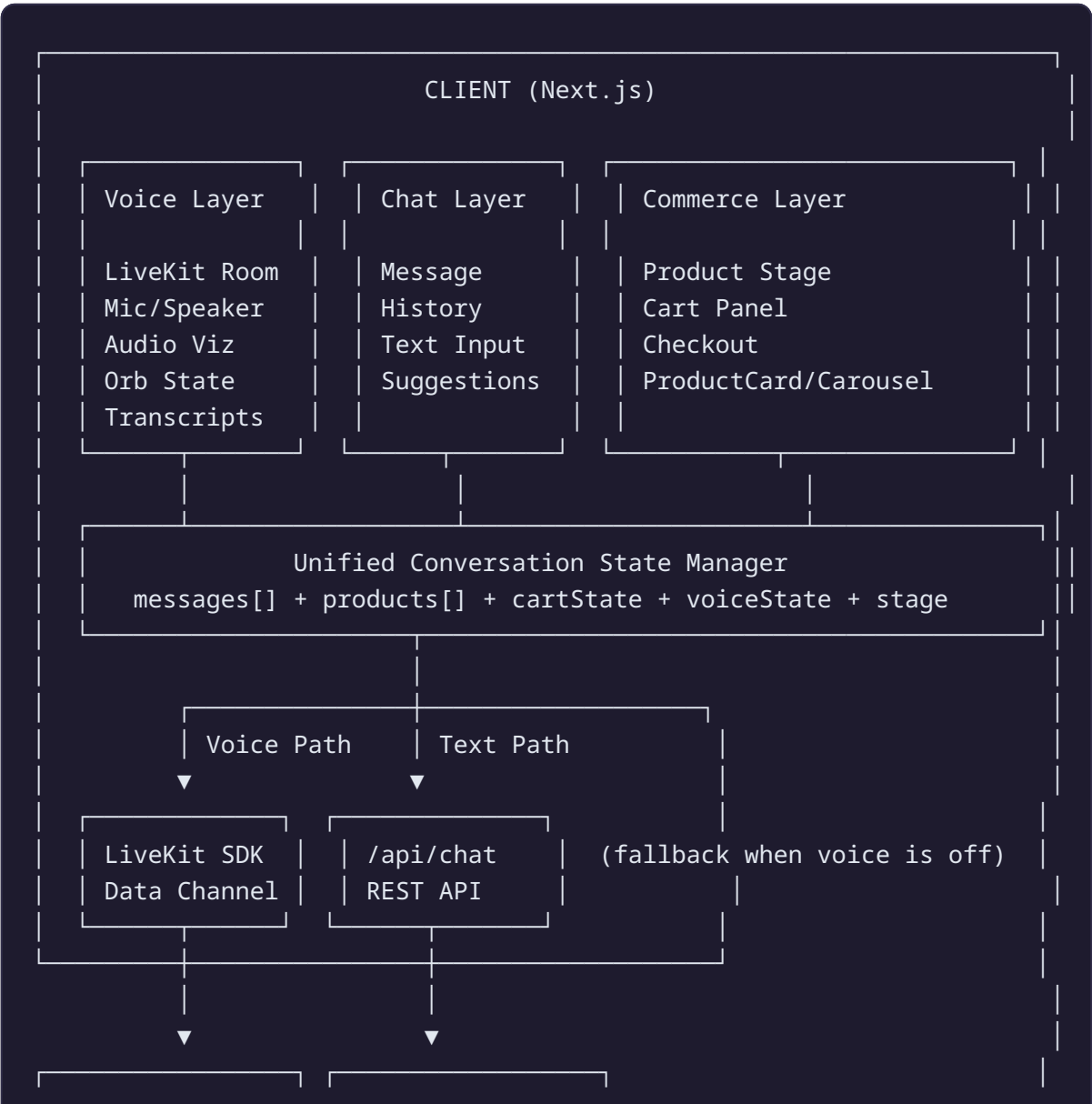
5.4 Cart View (Voice or Text)





6. System Architecture

6.1 Architecture Diagram





6.2 Data Flow: Voice Query

Timeline (ms):

```
0      User starts speaking
300    LiveKit transmits audio to Python agent
350    Agent forwards to Gemini Realtime
600    Gemini recognizes intent, issues tool call
650    Agent executes search_products() against Shopify
900    Shopify returns products
950    Agent sends products via Data Channel → Frontend
1000   Frontend renders product cards (user sees products!)
1100   Gemini composes voice response
1300   Agent voice streams back → Frontend plays audio
1300+  Transcript text streams word-by-word
```

Total time to visual: ~1 second
Total time to voice response: ~1.3 seconds

6.3 State Management

```
interface AppState {
  // Voice
  voiceConnected: boolean
  voiceState: 'idle' | 'connecting' | 'listening' | 'thinking' | 'speaking'
  micEnabled: boolean
  speakerEnabled: boolean

  // Conversation (unified)
  messages: ConversationMessage[]

  // Stage
  stageContent: 'welcome' | 'products' | 'product_detail' | 'cart' | 'checkout'
  products: Product[]
  selectedProduct: Product | null
  highlightedProductId: string | null // Which product agent is currently describing

  // Cart
  cartState: CartState | null
  cartId: string | null

  // Store
  credentials: StoreCredentials
}

interface ConversationMessage {
  id: string
  role: 'user' | 'assistant' | 'system'
  content: string
  source: 'voice' | 'text' | 'system'
  products?: Product[]
  cartState?: CartState
  checkoutUrl?: string
  timestamp: number
  streaming?: boolean // True while text is still streaming in
}
```

7. Technical Implementation Plan

Phase 1: Foundation (Week 1-2)

- [] Integrate LiveKit SDK into existing Next.js app
- [] Add `/api/token` route (from voice repo)
- [] Create Python voice agent with Shopify tool functions
- [] Implement data channel protocol (agent → frontend events)
- [] Build Orb component (CSS-based, state-driven)
- [] Add mic button to existing control bar
- [] Unified state manager for voice + text

Phase 2: Voice Commerce (Week 2-3)

- [] Wire tool calls to existing Shopify MCP/API
- [] Product cards appear on `products_found` data event
- [] Cart updates on `cart_updated` data event
- [] Checkout flow via `checkout_ready` event
- [] Real-time transcript streaming (user + agent)
- [] Stage layout with product area + transcript

Phase 3: Polish & UX (Week 3-4)

- [] Orb state animations (breathing, listening, speaking)
- [] Product card highlight when agent describes
- [] Stagger animations for card entrance
- [] Context-aware suggestion chips
- [] Mobile responsive layout
- [] Push-to-talk option
- [] Auto-disconnect on idle

Phase 4: Testing & Launch (Week 4-5)

- [] End-to-end voice QA (all product queries)
- [] Mixed modality testing (voice + text switching)
- [] Mobile browser testing (iOS Safari, Chrome Android)

- [] Performance profiling (audio latency, render time)
 - [] Accessibility audit (screen reader, captions)
 - [] A/B test: voice-first vs text-first default
-

8. Dependencies & Requirements

8.1 New Dependencies (Frontend)

```
{
  "@livekit/components-react": "^2.9.19",
  "@livekit/components-styles": "^1.2.0",
  "livekit-client": "^2.17.0",
  "livekit-server-sdk": "^2.15.0"
}
```

8.2 New Dependencies (Python Agent)

```
livekit-agents[images]
livekit-plugins-google
python-dotenv
httpx (for Shopify API calls)
```

8.3 Environment Variables

```
# Existing
GOOGLE_API_KEY=...
SHOPIFY_STORE_URL=jaguar-9969.myshopify.com
SHOPIFY_ACCESS_TOKEN=shpat_...

# New
LIVEKIT_URL=wss://halo-mcp-2tbjr4ch.livekit.cloud
LIVEKIT_API_KEY=API34HHb6JLeH6E
LIVEKIT_API_SECRET=...
```

8.4 Infrastructure

- LiveKit Cloud account (already provisioned)
- Python runtime for agent (can run on same server or separate)
- No additional Shopify changes needed

9. Open Questions

1. **Voice persona:** Should the agent's personality change based on occasion? (Playful for birthday, elegant for wedding)
2. **Multi-language:** LiveKit + Gemini support multiple languages. Should we enable this?
3. **Agent memory:** Should the voice agent remember preferences across sessions? ("Last time you bought for your wife...")
4. **Video/Vision:** The LiveKit agent supports camera. Should we enable "show me what you want to match" (point camera at outfit, get gift suggestions)?
5. **Agent hosting:** Self-hosted Python agent vs. LiveKit's hosted agent infrastructure?

10. Appendix: Component Inventory

New Components

Component	Description	Priority
VoiceProvider	Context provider wrapping LiveKit room + state	P0
Orb	Animated voice presence indicator	P0
VoiceControls	Mic/speaker/end buttons	P0
TranscriptStream	Real-time voice-to-text display	P0
Stage	Smart layout container for products/cart/detail	P0
OrderSummary	Checkout confirmation card	P0

Component	Description	Priority
<code>LiveCaption</code>	Overlay caption for mobile voice mode	P1

Modified Components

Component	Changes	Priority
<code>ChatInterface</code>	Add voice state, unified messages, stage layout	P0
<code>ProductCarousel</code>	Accept data channel events, highlight active	P0
<code>ProductCard</code>	Voice context indicator, quick add button	P1
<code>CartPanel</code>	Voice-triggered open/close, bottom sheet mobile	P1
<code>page.tsx</code>	Integrate VoiceProvider, Stage layout	P0

Preserved (No Changes)

Component	Reason
<code>StoreSwapModal</code>	Works as-is
<code>ChatMessage</code>	Text rendering unchanged
<code>/api/chat</code> route	Text fallback path
<code>/api/shopify/*</code> routes	Same API layer
<code>mcp-client.ts</code>	Reused by Python agent
<code>shopify-client.ts</code>	Reused for text fallback